

1P First steps

August 29, 2026

```
[1]: # FIRST STEPS IN PYTHON. VARIABLES, OPERATIONS, DATA SETS, PLOTS
      # Dr. M. Baron, Regression class, STAT 415-615

      # Vectors and simple operations

      x = [1, 3, 5, 7, 9]          # Create a vector and print it
      x
```

```
[1]: [1, 3, 5, 7, 9]
```

```
[2]: print(x)
```

```
[1, 3, 5, 7, 9]
```

```
[3]: print(x[1])      # Get the 2nd element of x.
      # In Python, indexing starts at 0, so x[1] is the 2nd element
```

```
3
```

```
[4]: # Arithmetic operations
      x+x              # Apparently, it is concatenation, not an addition!
```

```
[4]: [1, 3, 5, 7, 9, 1, 3, 5, 7, 9]
```

```
[5]: 3*x              # Same result, this is also concatenation!
```

```
[5]: [1, 3, 5, 7, 9, 1, 3, 5, 7, 9, 1, 3, 5, 7, 9]
```

```
[6]: squared_x = [y ** 2 for y in x]
      print(squared_x)
```

```
[1, 9, 25, 49, 81]
```

```
[7]: log_x = [__import__('math').log(y) if y > 0 else float('nan') for y in x]
      print(log_x)
```

```
[0.0, 1.0986122886681098, 1.6094379124341003, 1.9459101490553132,
2.1972245773362196]
```

```
[8]: # Basic statistics
mean_x = sum(x) / len(x)           # Mean
print(mean_x)
```

5.0

```
[25]: sd_x = (sum((y - mean_x) ** 2 for y in x) / len(x)) ** (1/2)
print(sd_x)                        # Standard deviation
```

2.8284271247461903

```
[27]: # This is too cumbersome! Instead, we'll use Python libraries.
# The first one is "numpy" = Numerical Python

# Numerical Python

import numpy as np                # Now we can use an abbreviation np

x = np.array([1, 3, 5, 7, 9])      # Define x as an array
x+x                               # Standard arithmetic on
↪ arrays
```

```
[27]: array([ 2,  6, 10, 14, 18])
```

```
[29]: 3*x
```

```
[29]: array([ 3,  9, 15, 21, 27])
```

```
[31]: x**2
```

```
[31]: array([ 1,  9, 25, 49, 81])
```

```
[33]: np.log(x)
```

```
[33]: array([0.          , 1.09861229, 1.60943791, 1.94591015, 2.19722458])
```

```
[35]: A = np.array([[1, 3, 5], [6, 8, 10]])      # Matrix 2x3
A
```

```
[35]: array([[ 1,  3,  5],
              [ 6,  8, 10]])
```

```
[37]: # Generate data, an array of Normal random numbers
Z = np.random.normal(0,1,100)      # mean, standard deviation, and sample size
```

```
[39]: A = np.matrix('1,3,5; 6,8,10')           # Same result using np.matrix
A
```

```
[39]: matrix([[ 1,  3,  5],
              [ 6,  8, 10]])
```

```
[41]: np.mean(Z)
```

```
[41]: -0.10603236278254234
```

```
[43]: np.std(Z)
```

```
[43]: 1.082144924956909
```

```
[45]: Z.mean() # Another way of calculating the sample mean
```

```
[45]: -0.10603236278254234
```

```
[51]: # Read data from an external file

# To point to the right folder, use os module
import os

print(os.getcwd()) # Get current working directory
```

C:\Users\baron\Documents\Teach\627 Statistical Machine Learning\Data

```
[49]: os.chdir("C:\\Users\\baron\\Documents\\Teach\\615 Regression\\Data") # Change
      ↪ the working directory
```

```
[53]: # Use pandas to read files
import pandas as pd
auto = pd.read_csv("Auto.csv") # Reading a comma-separated values file
```

```
[55]: # Find out the dimensions and variables of the data set
print(auto.shape) # Number of rows and columns
```

(397, 9)

```
[57]: print(auto.columns) # Variable names
```

```
Index(['mpg', 'cylinders', 'displacement', 'horsepower', 'weight',
       'acceleration', 'year', 'origin', 'name'],
      dtype='object')
```

```
[59]: print(auto.describe()) # Summary statistics
```

	mpg	cylinders	displacement	weight	acceleration	\
count	397.000000	397.000000	397.000000	397.000000	397.000000	
mean	23.515869	5.458438	193.532746	2970.261965	15.555668	
std	7.825804	1.701577	104.379583	847.904119	2.749995	
min	9.000000	3.000000	68.000000	1613.000000	8.000000	
25%	17.500000	4.000000	104.000000	2223.000000	13.800000	

50%	23.000000	4.000000	146.000000	2800.000000	15.500000
75%	29.000000	8.000000	262.000000	3609.000000	17.100000
max	46.600000	8.000000	455.000000	5140.000000	24.800000

	year	origin
count	397.000000	397.000000
mean	75.994962	1.574307
std	3.690005	0.802549
min	70.000000	1.000000
25%	73.000000	1.000000
50%	76.000000	1.000000
75%	79.000000	2.000000
max	82.000000	3.000000

```
[61]: # Look at the data as a spreadsheet
auto.head() # Show first 5 rows
```

```
[61]:      mpg  cylinders  displacement  horsepower  weight  acceleration  year  \
0   18.0           8           307.0           130   3504           12.0    70
1   15.0           8           350.0           165   3693           11.5    70
2   18.0           8           318.0           150   3436           11.0    70
3   16.0           8           304.0           150   3433           12.0    70
4   17.0           8           302.0           140   3449           10.5    70
```

	origin	name
0	1	chevrolet chevelle malibu
1	1	buick skylark 320
2	1	plymouth satellite
3	1	amc rebel sst
4	1	ford torino

```
[63]: # Refer to a particular variable in this dataset
print(auto['name'])
```

```
0      chevrolet chevelle malibu
1          buick skylark 320
2      plymouth satellite
3          amc rebel sst
4          ford torino
...
392      ford mustang gl
393          vw pickup
394      dodge rampage
395      ford ranger
396      chevy s-10
Name: name, Length: 397, dtype: object
```

```
[65]: print(auto['mpg'].mean()) # Mean of 'mpg'
```

23.51586901763224

```
[67]: print(auto['mpg'].describe()) # Summary statistics of 'mpg'
```

```
count      397.000000
mean       23.515869
std        7.825804
min        9.000000
25%       17.500000
50%       23.000000
75%       29.000000
max       46.600000
Name: mpg, dtype: float64
```

```
[69]: # PLOTS. Before you do anything with the data, look at them. Use the matplotlib
      ↪ library.
```

```
import matplotlib.pyplot as plt
```

```
plt.scatter(auto['weight'], auto['mpg']) # Scatterplot
```

```
# Axis labels, graph title, color
```

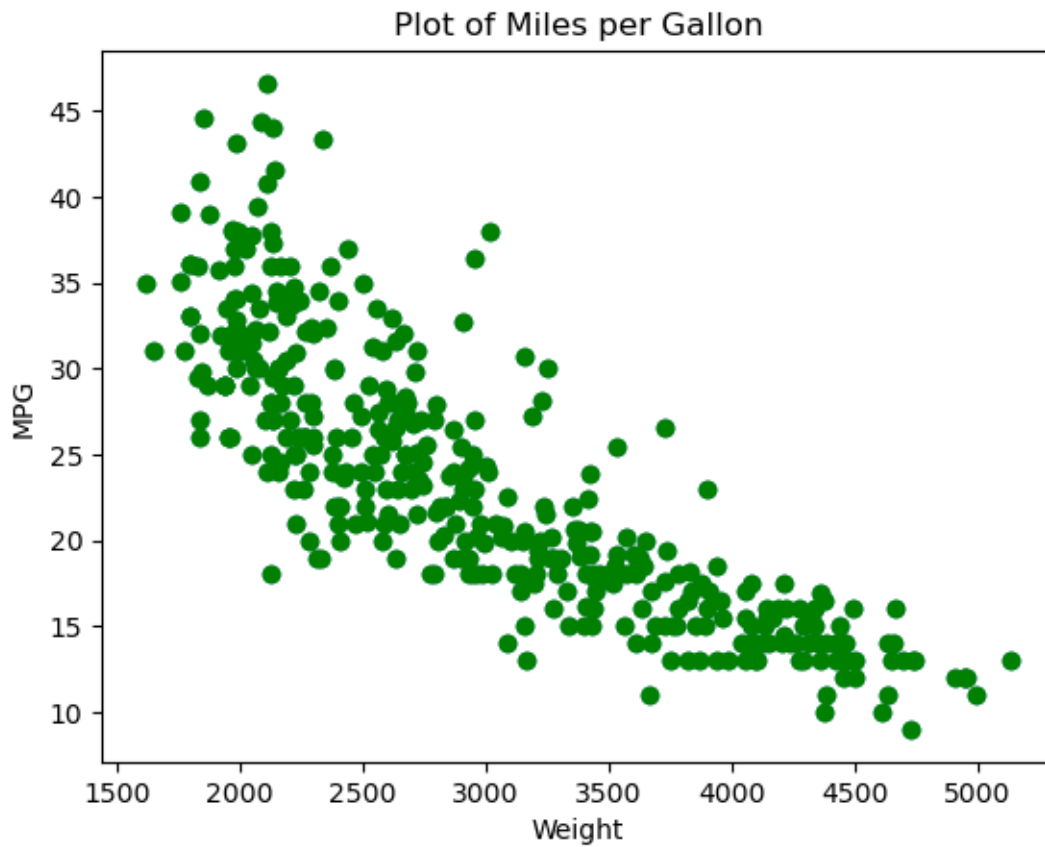
```
plt.scatter(auto['weight'], auto['mpg'], color='green')
```

```
plt.xlabel('Weight')
```

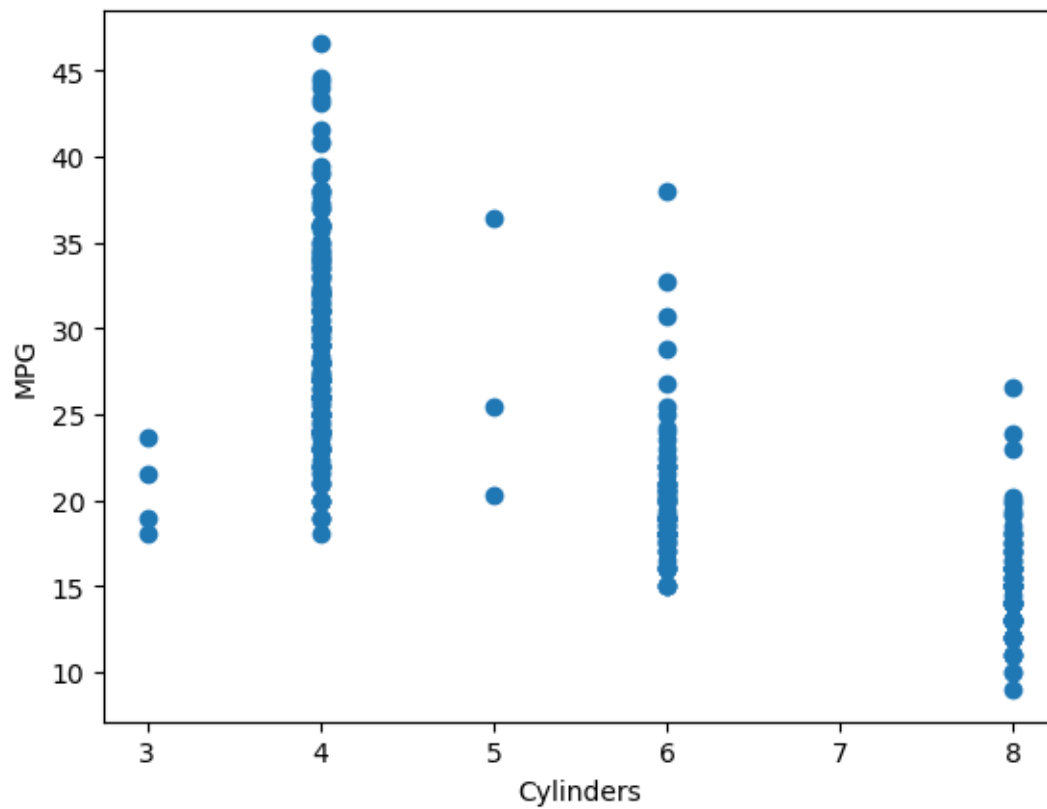
```
plt.ylabel('MPG')
```

```
plt.title('Plot of Miles per Gallon')
```

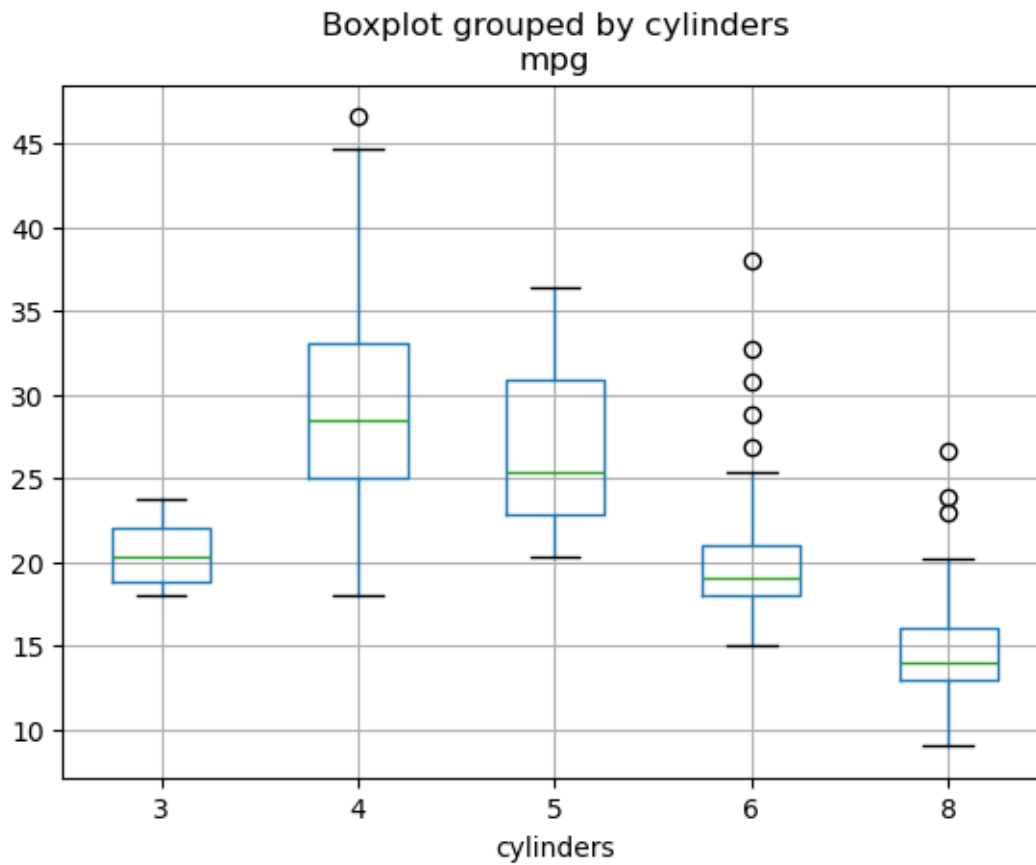
```
plt.show()
```



```
[196]: plt.scatter(auto['cylinders'], auto['mpg'])      # Another scatterplot
plt.xlabel('Cylinders')
plt.ylabel('MPG')
plt.show()
```



```
[186]: # Treat "cylinders" as a categorical variable => Python creates boxplots
auto['cylinders'] = auto['cylinders'].astype('category')
auto.boxplot(column='mpg', by='cylinders')
plt.show()
```



```
[202]: # SCATTERPLOT MATRIX
pd.plotting.scatter_matrix(auto[['mpg', 'weight', 'horsepower', 'year']],
    ↳figsize=(6,6))
plt.show() # Histograms on the diagonal, scatterplots of the corresponding
    ↳variables elsewhere
```